TaskFlow – JavaScript Project Documentation & Reflection

Project Overview

TaskFlow is a responsive task management web application created for the Week 5 JavaScript Project. Its objective is to help students and individual users organize work, track progress, and manage priorities through a simple interactive interface. The application stores tasks as JavaScript objects inside an array and uses LocalStorage so data remains available after refreshing the page.

Features Implemented

The application provides meaningful task-management features including adding, editing, deleting, and completing tasks. Users can search tasks by title, description, or category and filter them by pending, completed, or high-priority status. Tasks can also be sorted by creation time, priority, or due date. Form validation prevents empty titles and past due dates. A dynamic dashboard displays total, completed, pending, and high-priority counts, while a progress bar shows completion percentage. A light/dark theme toggle and responsive layout improve usability on different devices.

Technologies Used

HTML5 provides the semantic structure, forms, buttons, navigation, and content sections. CSS3 provides the responsive layout, cards, typography, theme variables, modal styling, and visual hierarchy. Vanilla JavaScript provides application logic, DOM manipulation, event handling, data processing, validation, and LocalStorage persistence.

JavaScript Concepts Used

The project demonstrates variables, constants, functions, arrow functions, arrays, objects, conditional statements, template literals, array methods such as filter(), sort(), and find(), DOM selection, dynamic element rendering, form submission, click/input events, keyboard events, and browser LocalStorage.

Challenges Faced

The main challenge was keeping the interface synchronized with the task data after every operation. Another challenge was validating user input while keeping the code reusable and readable.

Solutions Implemented

A centralized render() function rebuilds the task list and statistics whenever data changes. Reusable functions handle filtering, sorting, validation, modal behavior, and persistence. Event delegation is used for dynamically generated task buttons, which keeps event handling simple and efficient.

What I Learned

This project improved my understanding of how JavaScript connects application data with the DOM. I learned how arrays and objects can model real-world information, how events drive interactive applications, and how LocalStorage can preserve client-side data.

Future Improvements

Future versions could add user accounts, drag-and-drop Kanban boards, recurring tasks, notifications, cloud synchronization, and backend database support.

Project Output & Screenshot Checklist

The project is packaged with the complete HTML, CSS and JavaScript source. Before final submission, capture the running application in a desktop and mobile browser and place the images in the screenshots/ folder. The application itself is fully structured for responsive desktop, tablet and mobile views.

Required evidence	Status
HTML structure	Included
CSS styling	Included
JavaScript functionality	Included
Interactive features	Implemented and syntax-checked
README.md	Included
Documentation PDF	Included
Desktop/mobile screenshots	Capture from the live/local app before submission